Adaptive Candidate Selection for Period Recovery Under NISQ Noise

Robert Dohner

August 2026

Abstract

1 Introduction

RSA encryption was first proposed in 1978 as a way to ensure that electronic mail is both private and can be signed [7]. RSA encryption is based off of the notion that multiplying two extremely large prime numbers is easy, but factoring an extremely large number into two prime numbers is a difficult process, or as described by the creators, "a trap-door one-way function" [7]. Since this time, RSA has become one of the main ways of encryption, used in many applications where the security of digital data is required [1]. However, with the advancement of quantum computing, there is a real possibility that sometime in the near future that anything encrypted with RSA will become vulnerable. This so called "Quantum Day", the hypothetical day when quantum computers will be able to break RSA encryption, is hard to predict, as the tools required for this task are still being created and developed [5].

Shor's Algorithm is the main focal point of breaking RSA encryption. It is an algorithm that uses quantum methods and order finding in order to bypass the difficulty that factoring presents [8]. Despite advancements in Shor's Algorithm that require fewer qubits [11], the scale of quantum computer that is required to break RSA encryption is currently unavailable. The other problem with the algorithm is that it is highly susceptible to Noisy Intermediate-Scale Quantum (NISQ) technology, which all quantum computers currently are. Noise is currently a major problem for quantum computers that possess 50 qubits or more [6]. As such, methods to help mitigate the noise problem are currently being investigated. One of these methods, an approach that classifies features of the noisy distribution that is provided by Shor's Algorithm in order to figure out if the process is recoverable despite the noise [10]. We will examine this methodology

and expand upon the novel approach.

2 Background

The main concept of Shor's Algorithm is the quantum order finding methodology. Given an integer N and a random integer that is coprime to N, defined as a, Shor's algorithm attempts to find the smallest positive integer r, where:

$$a^r \equiv 1 \pmod{N} \tag{1}$$

Once r, also known as the hidden period, is obtained, it is relatively simple to extract the real factors using Euclid's greatest common divisor algorithm.

$$\begin{aligned} p &= GCD(a^{r/2} - 1, N) \\ q &= GCD(a^{r/2} + 1, N) \end{aligned} \tag{2}$$

This order information is encoded in the measurement distribution of a t-qubit phase-estimation, also known as precision, register. With the ability to write $Q = 2^t$ unique bitstrings which can hold any value of y that is between 0 and $Q - 1$ for the measured register value, the output is a probability distribution $p(y)$. In an ideal, noiseless environment, $p(y)$ should be concentrated and evenly spaced, with the concentrations near the locations of $y \approx Q * s/r$, where $s = 0, ..., r - 1$ [10].

The problem is that when noise is taken into account, the ideal structure becomes distorted, thus muddling the hidden period. Sometimes, this leads to situations where the correct hidden period is nonrecoverable.

More to add...

3 Methods

Dataset Collection

The method to collect datasets will fall into two categories: datasets from the Aer simulator from Qiskit and datasets from the sample backends provided by IBM. The Aer simulator is a high performance simulator designed to mimic a quantum computer, complete with the ability to replicate the noise that a normal NISQ quantum computer will face in the real world [3]. The fake backends from IBM are "built to mimic the behaviors of IBM systems using system snapshots" [2].

Datasets created from either a simulator or a fake backend are built from four variables. N is the number we are trying to find the factors for. In this case, N belongs to a family of Mersenne / powers-of-two family, $N = 2^n - 1$. The reason for this is to allow for a bit-shifting trick to be implemented in order to lower the number of quantum gates required. Take any $x \in \{0, 1, ..., N-1\}$, which is written in bits as:

$$x = \sum_{i=0}^{n-1} b_i 2^i, b_i \in 0, 1 \tag{3}$$

Multiplying by 2 shifts every bit up one place:

$$2x = \sum_{i=0}^{n-1} b_i 2^{i+1} = b_{n-1} * 2^n + \sum_{i=0}^{n-2} b_i 2^{i+1}, b_i \in 0, 1 \tag{4}$$

We are assuming that for all N, $N = 2^n - 1$. This gives us $2^n = N + 1$. This means that 2^n is one more than a multiple of N, which is the same as $\equiv 1(modN)$. We also have Equation (1), which leads to:

$$2^n \equiv 1 \pmod{N} \tag{5}$$

In modulo arithmetic, we can swap 2^n for 1 in modulo N, giving us the following:

$$2x \pmod{N} = b_{n-1} + \sum_{i=0}^{n-2} b_i 2^{i+1}, b_i \in 0, 1 \tag{6}$$

It is this exact special case in modulo arithmetic that our custom quantum circuit attempts to take advantage of, which allows us to reduce the number of quantum gates used. The values of N we are using are $N \in \{3, 7, 15, 31, 63, 127\}$.

For our other three variables, a from Equation (1) is such that $a \in \{2, 4, 6, 8\}$. Variable t is the number of qubits in our control register, which holds the phase-estimation output y and can either be 8 or 10. The approximate QFT degree controls if some of the gates in the inverse quantum Fourier transform gets dropped. This variable trades a small amount of precision for a reduction of the size of the circuit. We use three values for this, $approxDegree \in \{0, 1, 2\}$ An approximate QFT degree of 0 means no quantum gates are dropped, while a degree of 1 or 2 will progressively drop more and more of the least-impactful rotation gates.

All runs use 4000 shots and transpilation with Qiskit optimization level 1 [10]. For the Aer simualtor, five runs are made, each with differing levels of noise. The first run is made without any noise, an ideal quantum computer run. This run would be removed from the final overall statistics due to the fact that the final statistics examines how noise interacts with recoverability. The second run adds a noise level of 0.08 to all single qubit gates. The third run adds a noise level of 0.16 to all two qubit gates. The fourth run adds a

noise level of 0.15 to the readout error of the circuit. The final run combines all of the noise from above into one run. For the backend simulations, the fake backends used are FakeKingston, FakeBoston, FakeTorino, and FakePittsburgh to coincide with the real backends that the Yang-Markidis paper used.

Due to time constraints, some modifications to the datasets had to be made. The Yang and Markidis paper ran some runs on ibm_miami. However, those runs only represented 1.5% of the total runs and due to time constraints and the fact that particular fake backend consistently ran the slowest, that particular fake backend was dropped. Because running the circuits were taking a long time on the fake backends, another change was that the circuit was transpiled multiple times across multiple seeds and then the smallest circuit was chosen in order to be ran on the fake backends. Also due to time constraints, $N = 127$ was dropped for the fake backends.

Features for Recoverability

The ideal output of quantum order finding has a characteristic peak-comb structure [10]. Because, ideally in a real quantum simulation, we would not know the actual true hidden order for the N we are provided, we have to investigate features that we can deduce from the precision-register distribution.

The Yang-Markidis paper used four features to measure recoverability, two general statistical features involving the distribution level and two explicitly built features that are post-processing-aware and built around their recoverabilty definition. The first recoverability feature is autocorrelation peak strength (A_{peak}), which describes how flat the distribution is. The larger the value we get, the more that the noisy distribution preserves the repeated-spacing structure that a noiseless order finding would possess. We can define this with the following equation set of equations, where $u(y) = 1/Q$, denoting a uniform distribution:

$$\begin{aligned} q(y) &= p(y) - u(y) \\ A(\ell) &= \sum_y q(y)q(y+\ell) \\ A_{peak} &= max_{t\neq 0} A(\ell)/A(0) \end{aligned} \tag{7}$$

The second feature is normalized entropy (H_{norm}), which represents if the distribution exhibits repeated peak organization. The larger the value, the more the probability mass is more broadly spread out, which means it is less concentrated around the peaks. This is defined by the following equation:

$$H_{norm}(p) = (-\sum_y p(y)log(p(y)))/logQ \tag{8}$$

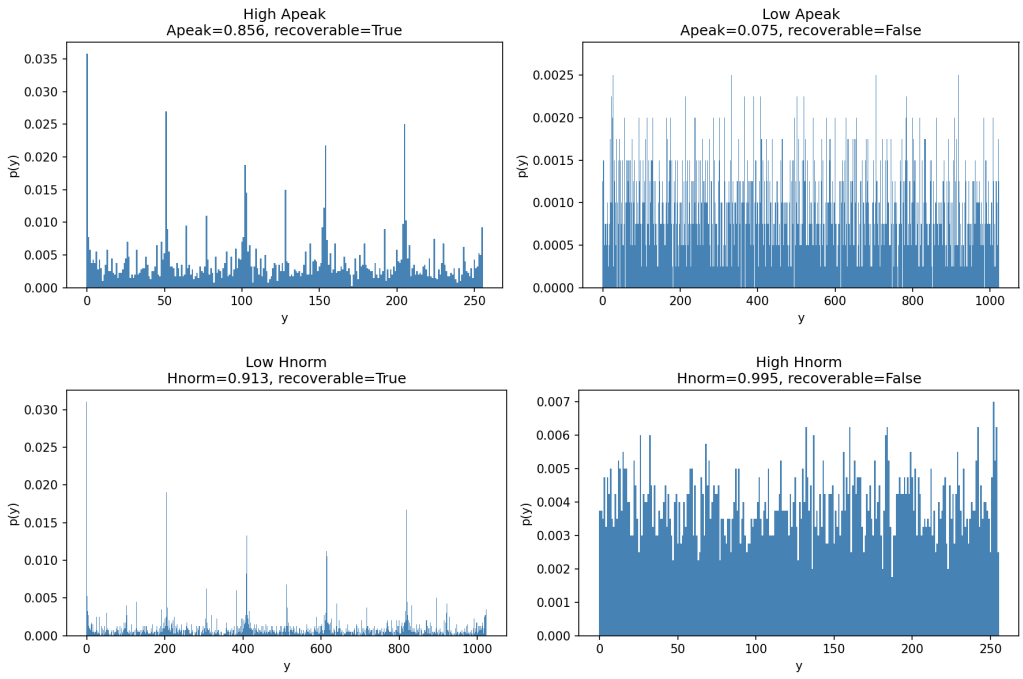

Figure 1: Examples of high and low Apeak and Hnorm

The two post-processing-aware variables rely on finding r_0, which is a verified candidate denominator found through continued fractions. A continued fraction is the expression of the form [4]:

$$a_0 + \cfrac{1}{a_1 + \cfrac{1}{a_2 + \cfrac{1}{a_3 + \dots}}} \tag{9}$$

The reason why continued fractions are so important to RSA encryption is because equation (1) can eventually be rewritten to a point where [9]:

$$\begin{aligned} ed &\equiv 1 \pmod{N} \\ e/N &\approx k/d, \text{for some integer k} \end{aligned} \tag{10}$$

where d is a public exponent and e is a private exponent. If d is small, then continued fractions can be used to approximate e/N.

For Shor's algorithm, continued fractions are used in the final classical post-processing step after we receive a noisy phase-estimation. Continued fractions are the tool used to round that phase-estimation to the nearest fraction with a small denominator, thus leading to the hidden period r [8].

For each verified candidate denominator, r_0, we can calculate the probability mass assigned to each verified candidate denominator through [10]:

$$m(r_0) = \sum_{\substack{y : r_0(y) = r_0 \\ a^{r_0} \equiv 1 \pmod{N}}} p(y) \tag{11}$$

Using these probability mass, we can find the total verified mass, which is simply $M_{ver} = \sum_{r_0} m(r_0)$. The largest $m(r_0)$ can be denoted with M_1 while the second largest can be denoted with M_2. This total verified mass is used in two post-processing-aware features, the first of which is the dominant verified mass fraction, which is defined as such:

$$M_{1,frac} = M_1 / M_{ver} \tag{12}$$

This is a representation of largest candidate order dominating the total verified mass. The next feature we can use is called the verified margin fraction, which is as follows:

$$\Delta_{ver,frac} = (M_1 - M_2) / M_{ver} \tag{13}$$

This is a representation of the separation between the largest candidate order and the second largest.

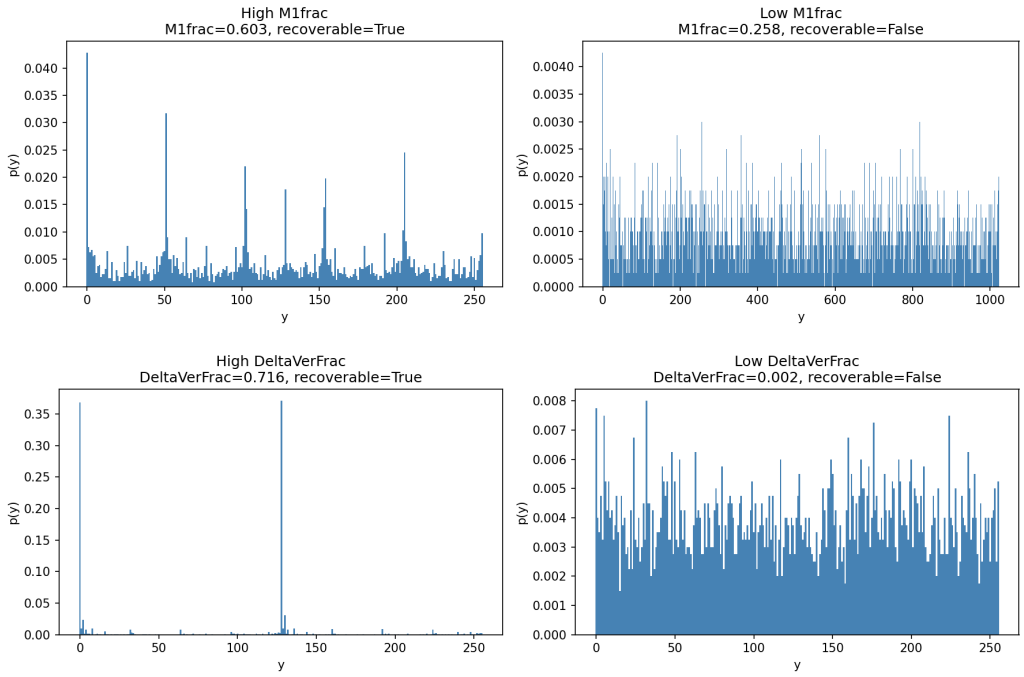

Figure 2: Examples of high and low $M1_frac$ and $\Delta_{ver,frac}$

In addition to these features, we added three more features. Those features are skewness, kurtosis, and number of peaks, which are all standard characteristics of a distribution. Skewness represents how

asymmetric a distribution is around its mean. It is a number that shows whether or not a histogram is right-skewed, leading to a skewness of greater than zero, or left-skewed, with a skewness less than zero. Given that μ is the mean of the distribution and σ is the standard deviation, skewness can be found with the following equation:

$$\text{skewness} = \frac{\sum_y p(y)(y-\mu)^3}{\sigma^3} \tag{14}$$

Kurtosis, on the other hand, measures how much of the distribution's variance comes from its tails, or the extreme values. This can be measured with the following:

$$\text{kurtosis} = \frac{\sum_y p(y)(y-\mu)^4}{\sigma^4} - 3 \tag{15}$$

The number of peaks is measured through finding how many local maxima appear in the distribution. We use the the find_peaks function from scipy.signal Python package in order to come up with this number.

4 Results

Simulator Results

For our simulator runs, our results were surprisingly very close to the paper's runs on the real quantum computers. The percentage that was recoverable was 59.8%. The discrepancy could be explained in the dataset. We did more runs at lower numbers and at more precise approximation degrees while the paper, which did not define how many runs they did of each N, did a more skewed dataset in terms of t and approximation degree. There is a good chance they also skewed their values of N more towards the higher side.

Figure 3 shows the four features from the paper grouped into their categories: distribution level and post-processing-aware. We can see that at high entropy and low autocorrelation peak strength, the distributions become more unrecoverable. For the post-processing-aware features, if both values are low, then the distribution tends to become more nonrecoverable.

Figure 4 shows all of our measured features and how those values stack against being recoverable or not recoverable. For most of the figures, most of the not recoverable models share similar attributes. They have a low A_{peak}, $M1_{frac}$, and $\Delta_{ver,frac}$ along with a high H_{norm}. The skewness and kurtosis tends to be symmetrical, uniform distribution. However, having one of these attributes does not necessarily mean that the distribution is nonrecoverable.

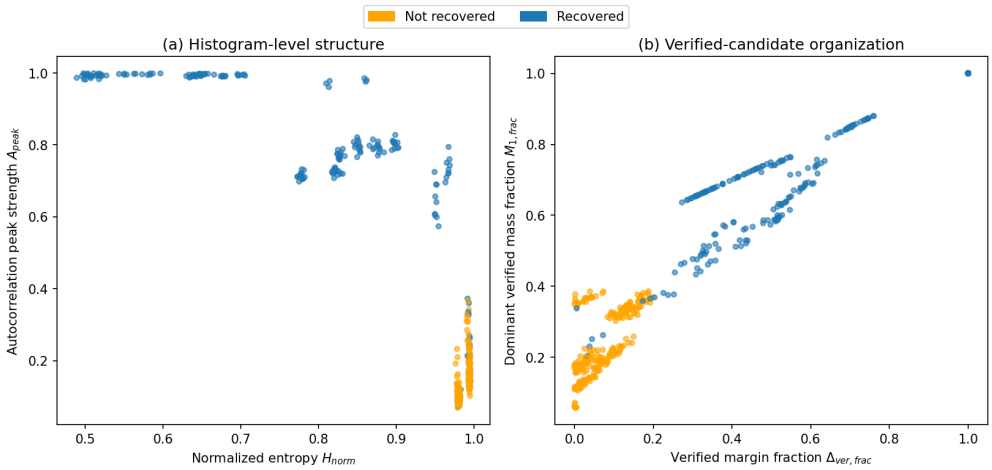

Figure 3: Grouped features for the simulator runs

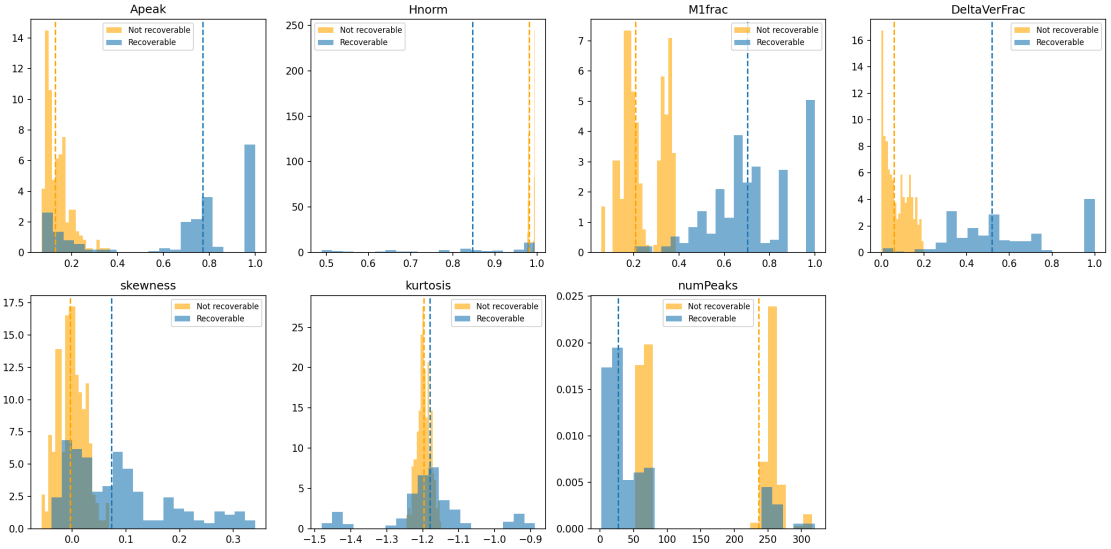

Figure 4: Individual features for the simulator runs

In order to figure out which of these attributes factor the most in terms of importance, we have statistical tools to help us out. The first one is a single feature AUROC, or area under the receiver operating characteristic curve. We use this in order to compare recoverable runs to nonrecoverable runs. The goal is to figure out the probability that a recoverable run would have a higher (or lower for some features) feature score than a nonrecoverable run. A score of 1.0 would mean that the feature is always able to predict the difference between a recoverable run and a nonrecoverable run. A score of 0.0 would mean the complete opposite. A score of 0.5 would mean that the feature is no better than a random guess.

Table 1: Ranking of the seven selected features from the simulator by single-feature AUROC for recoverability.

Rank	Metric	AUROC
1	Dominant verified mass fraction $M_{1,frac}$	0.990
2	Verified margin fraction $\Delta_{ver,frac}$	0.987
3	Autocorrelation peak strength A_{peak}	0.867
4	Normalized entropy H_{norm}	0.860
5	Number of Peaks	0.855
6	Skewness	0.826
7	Kurtosis	0.596

With our dataset from the simulator, there are two features that clearly stand out above the rest. The two post-processing-aware features were clearly better than the other features at predicting whether or not a distribution would be recoverable. The other features, outside of kurtosis, were all roughly similar in strength. Kurtosis was just a little better than a random coin toss at predicting whether or not a feature would be recoverable. These results would further indicate that recoverability is governed by how the probability mass is distributed among verified candidates. On the other hand, the overall structure of the distribution, while it does seem to play a small role in how recoverability is affected, seems to have less correlation than the probability mass of verified candidates.

The AUROC scores are able to show which feature can predict recoverability when that feature is in isolation. However, in order to look at how the features provide information in tandem with the rest of the features, a random forest classifier can be used. Our random forest classifier will be trained on four folds and then evaluated on the held out fold. One feature will be randomly permuted at a time. Then, the decrease in AUROC score is recorded. The greater the decrease in AUROC score, the more that the classifier relies on that feature.

Figure 5 shows that the classifier favors verified margin fraction the most, despite it having a slightly lower individual AUROC score than its post-processing-aware counterpart, the dominant verified mass fraction. This means that once we know what the verified margin fraction is, there is a good chance that we will know what the recoverability of the distribution will be. While we can derive what the recoverability will be from

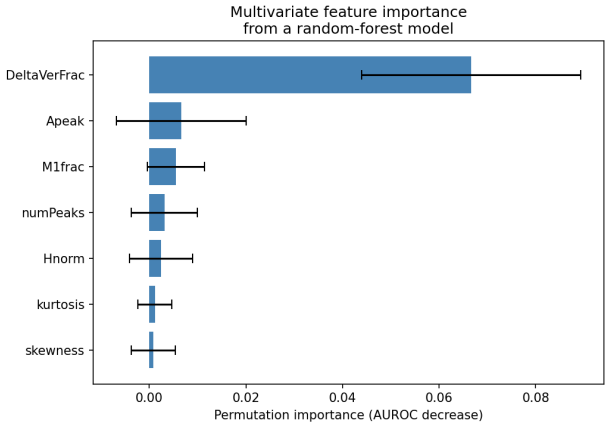

Figure 5: Permutation importance from a random-forest classifier trained on the seven selected recoverability features

the other features, they become less important once we know what the verified margin fraction is.

Backend Results

When we created our datasets from the fake backends, an interesting occurrence happened. While our simulator datasets had a recoverability rate of 59.8%, our fake backend dataset had a recoverability rate of 86.7%. Part of this is due to $N = 127$ not being evaluated, but even if we throw that out for the simulator, the recoverability rate for the simulator is still at a 64.8%. Table 2 shows where the discrepancy happens.

Table 2: Simulator vs Fake backend

N	Simulator Recoverabilty	Simulator Count	Backend Recoverability	Backend Count
3	100%	60	100.0%	48
7	100%	90	100.0%	72
15	61.1%	90	100.0%	72
31	40.0%	120	68.8%	96
63	48.3%	120	78.1%	96
Overall	64.8%	480	86.7%	384

When $N = 3$ or $N = 7$, the recoverability of the model is 100%. This is due to the fact that the circuit is small enough to withstand the noise of both the simulator and the fake backends. However, as N increases for the simulator, the noise begins to overpower the circuit, causing massive spikes in nonrecoverability. However, that same spike does not seem to happen with the fake backends. In fact, the fake backends have a much higher rate of recoverability than the simulator and much higher than the 45.6% from the Yang-Markidis paper.

A few reasons can explain this. The first is that the noise level on the simulator is much higher than the noise level of the fake backends. This was done in order to better mimic the percentage obtained from

the real backends. The second is due to the fact that $N = 127$ is removed. The third could be in how the paper created its own datasets. We already know that it was not an even distribution between all of the variables. How they distributed the variables were never fully explained and only some totals per variable were provided. The fourth reason could be that the included seed optimizations caused the recoverability rates to increase.

When we look at the the grouped features in a scatter plot in Figure 6, we see something even more interesting. When compared to Figure 3, we see that the shapes roughly correlate with each other. Most of the recoverable runs that had similar statistics between both the simulator and the fake backend were still recoverable. However, when we compare the nonrecoverable runs, runs that had low A_{peak} and high H_{norm} were all of a sudden more recoverable on the fake backends. The same holds true for some runs with low $M_{1,frac}$ and low $\Delta_{ver,frac}$.

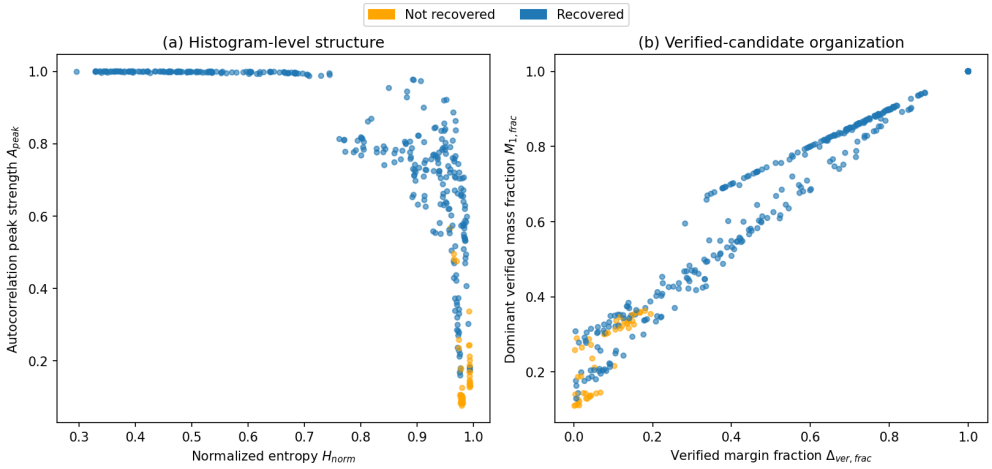

Figure 6: Grouped features for the fake backend runs

5 Discussion and Future Works

References

[1] Dan Boneh. Twenty years of attacks on the RSA cryptosystem. *Notices of the American Mathematical Society (AMS)*, 46(2):203–213, 1999.

[2] IBM Quantum. Fake provider — qiskit ibm runtime documentation. `https://quantum.cloud.ibm.com/docs/en/api/qiskit-ibm-runtime/fake-provider`. Accessed: 2026-08-09.

[3] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. Quantum computing with qiskit, 2024.

[4] A. Ya. Khinchin. *Continued Fractions*. University of Chicago Press, Chicago, 3rd edition, 1964. Originally published in Russian, 1935.

[5] Michele Mosca. Cybersecurity in an era with quantum computers: will we be ready? Cryptology ePrint Archive, Paper 2015/1075, 2015.

[6] John Preskill. Quantum computing in the nisq era and beyond. *Quantum*, 2:79, August 2018.

[7] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.

[8] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, October 1997.

[9] Michael J. Wiener. Cryptanalysis of short RSA secret exponents. *IEEE Transactions on Information Theory*, 36(3):553–558, 1990.

[10] Qingxin Yang and Stefano Markidis. When noisy quantum order finding remains recoverable for shor's algorithm, 2026.

[11] Christof Zalka. Shor's algorithm with fewer (pure) qubits, 2006.